SecAlertBench: Evaluating Large Language Models for Tier-1 Alert Triage in Security Operations Centers

Anonymous authors

ABSTRACT

Enterprise security operations centers (SOCs) are heavily affected by alert fatigue, as analysts must triage massive volumes of noisy alerts while identifying the small fraction that truly warrant escalation. Although large language models (LLMs) have shown strong potential in cybersecurity, their capability for Tier-1 alert triage in real-world SOC environments remains unclear. To address this gap, we construct SecAlertBench, a benchmark built from alerts collected from the SOCs of three large enterprises. After standardization and annotation, SecAlertBench contains 8,322 alert logs spanning 241 alert types, with binary labels of Attack and Non-Attack. Using this benchmark, we systematically evaluate 16 LLMs for Tier-1 alert triage. The results show that current LLMs already exhibit promising but limited triage capability, achieving an average TPR of 79.71% and an average F1-score of 70.92%, while still suffering from a high average FPR of 44.13%, indicating a substantial trade-off between attack detection and false-positive control. Beyond aggregate performance, we further conduct multidimensional analyses of LLM behavior, such as decision consistency and sensitivity to experimental configurations, among other aspects. Based on these results, we summarize the key limitations and bottlenecks of LLMs in current SOC Tier-1 alert triage. Overall, our findings suggest that LLMs are promising assistants for Tier-1 alert triage, but substantial improvements are still required before they can be reliably deployed as standalone solutions in real-world SOCs.

CCS CONCEPTS

• **Security and privacy** → *Network security*.

KEYWORDS

Security Operations Centers, Alert Triage, Large Language Models, Alert Fatigue, Cybersecurity Benchmark

ACM Reference Format:

Anonymous authors. 2018. SecAlertBench: Evaluating Large Language Models for Tier-1 Alert Triage in Security Operations Centers. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 16 pages. https://doi.org/XXXXXXX.XXXXXXX

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Network security devices, including firewalls, intrusion detection and prevention systems (IDS/IPS) [5, 20], web application firewalls (WAF) [24], and security information and event management (SIEM) platforms, generate a massive number of security alerts on a daily basis. These alerts encompass a wide range of event types, from potential attack indicators to routine system activities. However, in real-world deployments, the vast majority of alerts, often exceeding 90%, are false positives or benign events that do not require further investigation [1]. The overwhelming volume of low-value alerts makes it extremely challenging for security analysts to process them in a timely manner. As a result, Security Operations Centers (SOCs) frequently suffer from alert fatigue, where critical threats may be overlooked or delayed due to the excessive noise in alert streams [19, 34].

To cope with this issue, the first stage of SOC workflows, commonly referred to as Tier-1 alert triage, focuses on rapidly filtering and categorizing alerts to reduce noise before deeper analysis. Despite its importance, effective alert triage remains a challenging problem. Traditional approaches, such as rule-based systems and machine learning methods, struggle to balance detection capability and noise reduction. Rule-based systems often rely on coarse-grained matching over structured fields (e.g., ip addresses and ports), while neglecting the richer semantic information in payloads. Meanwhile, machine learning [2, 4] and deep learning methods [8, 17, 32, 36] heavily depend on feature engineering and lack the ability to generalize to evolving or unseen threats. Consequently, there is an urgent need for more robust and adaptive solutions to improve early-stage alert triage and alleviate analyst burden in SOC operations [23, 28].

While alert triage poses significant challenges for manual efforts and traditional machine learning–based techniques, we argue that it represents a sweet spot for Large Language Models (LLMs) [21, 31] for two reasons. First, LLMs can flexibly perform semantic-level interpretation, allowing them to abstract away implementation-level variations that often hinder the development of manual triage rules or prior-knowledge-guided model training. This capability enables more reliable and automated triage. Second, Tier-1 triage does not require the same level of precision as downstream attack detection tasks. While false negatives (i.e., missed attack events) should be minimized and reducing false positives is desirable, it is acceptable to pass some false positives to downstream analysis. This relaxed precision requirement makes LLMs well-suited for this setting, even if off-the-shelf models may not match the precision of well-defined rules or specialized models trained specifically for attack detection.

Nevertheless, while prior work has examined LLM performance across various security applications—such as network security knowledge assessment [35], code analysis [9], and phishing detection [14]—little attention has been given to their effectiveness in assisting with Tier-1 alert triage, despite its strong potential. More specifically, we identify two key gaps that need to be addressed.

Research Gaps. There are two critical research gaps:

- **Lack of a dedicated dataset for Tier-1 alert triage.** Existing evaluation datasets for LLMs in cybersecurity are primarily designed to assess broad security knowledge or other specialized tasks, rather than the practical problem of Tier-1 alert triage in SOC operations [15, 30, 38]. Meanwhile, datasets historically used for alert-based detection are often either unavailable for public research or limited to raw network traffic data (e.g., pcap files), rather than the structured alert logs required for alert triage analysis [27, 32, 40]. As a result, there is still no dedicated dataset that captures the real-world characteristics of high-volume, noisy, and semantically rich alerts encountered in early-stage SOC triage.
- **Lack of systematic evaluation of LLMs for Tier-1 alert triage.** Although LLMs have shown promise in various cybersecurity tasks, their effectiveness in Tier-1 alert triage remains poorly understood. In particular, there is no comprehensive evaluation comparing different categories of LLMs on this task, nor a systematic analysis of how key experimental factors, such as prompt design, model quantization, and other inference settings, affect performance. Consequently, the practical utility and limitations of LLMs in Tier-1 alert triage remain insufficiently characterized.

Our Work. As shown in Figure 1, to bridge these gaps, we present SecAlertBench, a benchmark and empirical study for evaluating LLMs on Tier-1 alert triage in real-world SOCs. We construct SecAlertBench from large-scale network-side alerts collected from the production SOC environments of three large enterprises, supplemented with selected public security datasets to improve attack coverage. After cleaning, normalization, anonymization, and expert annotation, SecAlertBench contains 8,322 annotated alert logs spanning 241 alert types, with binary labels of *Attack* and *Non-Attack*. The benchmark is designed to reflect the core Tier-1 triage task: deciding whether an alert should be escalated as attack-related or filtered as non-attack noise based on the alert evidence available at triage time.

Using SecAlertBench, we systematically evaluate 16 representative LLMs for Tier-1 alert triage. Our results show that current LLMs exhibit promising attack-detection capability, but remain insufficiently reliable for standalone SOC deployment. On average, the evaluated models achieve a TPR of 79.71%, indicating that they can identify a substantial portion of attack-related alerts. However, this comes with a high FPR of 44.13%, meaning that many benign or business-related alerts are still incorrectly escalated. Although the average F1-score reaches 70.92%, the high false-positive rate reveals a key practical bottleneck: strong attack detection alone is not enough to alleviate alert fatigue. Beyond aggregate performance, we further analyze LLM behavior from four aspects: response determinism, configuration sensitivity, reasoning-decision alignment, and alert-type generalization. We find that LLM decisions become less stable as decoding randomness increases, while prompt design, especially few-shot prompting, substantially affects the trade-off between attack detection and false-positive control. We also observe that correct final labels do not always come with faithful reasoning, and that model performance varies significantly across alert types, with semantically ambiguous alerts remaining particularly challenging.

Finally, we conduct a manual failure attribution analysis to summarize the main limitations and bottlenecks of current LLMs in Tier-1 alert triage. We analyze both efficiency and effectiveness bottlenecks, showing that current LLMs lack sufficient throughput for million-scale daily SOC workloads and still suffer from excessive false positive escalation. Our case studies further reveal that high false positives are mainly caused by missing enterprise context, semantic ambiguity in benign business traffic, and the security-sensitive behavior of LLMs. These findings highlight that LLMs should currently be viewed as assistive components rather than standalone Tier-1 triage engines.

Contributions. We summarize our key contributions as follows:

(1) **Dedicated Tier-1 Alert Triage Benchmark.** We construct SecAlertBench, a real-world benchmark designed specifically for evaluating LLMs on SOC Tier-1 alert triage. The benchmark contains 8,322 expert-annotated alert logs spanning 241 alert types, providing a realistic and task-specific foundation for studying LLM-based alert triage.
(2) **Systematic LLM Evaluation.** We conduct a comprehensive performance evaluation of 16 representative LLMs under a unified single-alert classification setting, focusing on their ability to detect attack-related alerts while controlling false positives. Beyond overall performance, we further analyze model behavior from four aspects: response determinism, configuration sensitivity, reasoning-decision alignment, and alert-type generalization. Our results show that current LLMs demonstrate promising attack-detection capability, but still suffer from high false-positive rates.
(3) **Limitations and Bottlenecks Analysis.** We analyze the practical bottlenecks that hinder the deployment of LLMs for Tier-1 alert triage. Our results show that current LLMs face both insufficient throughput for million-scale SOC workloads and excessive false positives caused by missing operational context, semantic ambiguity, and security-sensitive decision behavior.

2 BACKGROUND AND RELATED WORK

2.1 SOC Alert Triage

SOC alert triage is a staged process for handling security alerts generated by monitoring systems such as IDS, IPS, WAF, endpoint sensors, and SIEM platforms. In a typical SOC workflow, Tier-1 analysts perform the first level of alert review, where they quickly examine the available alert evidence, filter obvious false positives, classify alert severity, and decide whether an alert should be closed or escalated. Alerts that cannot be confidently resolved at Tier-1 are passed to Tier-2 analysts, who conduct deeper investigation by correlating multiple logs, checking asset and user context, consulting threat intelligence, and determining whether the alert reflects a real security incident. For complex, high impact, or persistent threats, Tier-3 analysts take over advanced analysis and response, including malware analysis, forensic investigation, threat hunting, root cause analysis, containment, remediation, and detection rule improvement. Therefore, Tier-1 triage plays a critical role in reducing alert noise and ensuring that higher tier analysts focus their effort on alerts that truly require deeper investigation.

2.2 Large Language Models in Cybersecurity

Most LLMs are built on the transformer architecture [33]. The key innovation of this architecture lies in the self-attention mechanism,

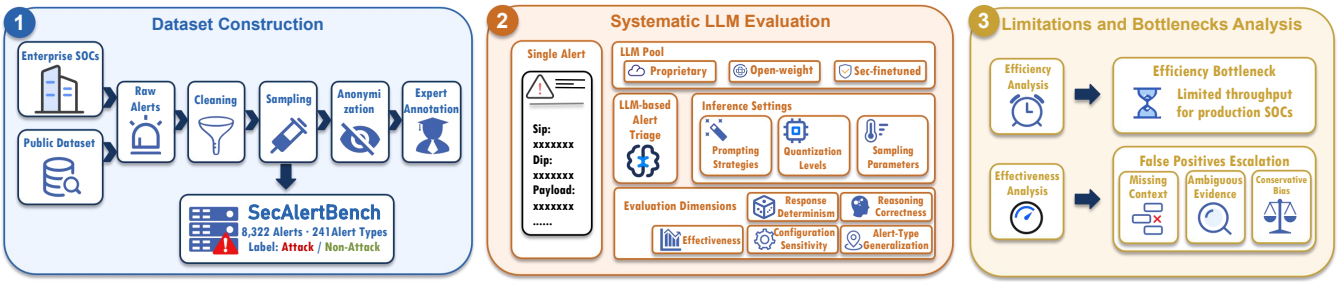

Figure 1: Overview of our study.

which allows the model to dynamically evaluate the relative importance of different words in the input text, thereby capturing long-range dependencies and complex contextual relationships. Pre-trained on massive and diverse text corpora through the next-word prediction task, these models develop a strong ability to understand meaning in language [39]. This emerging capacity for deep semantic understanding enables LLMs to interpret the implications and intentions behind semi-structured texts, such as the payloads in network alerts. The powerful text understanding and reasoning capabilities of LLMs have catalyzed a wave of innovation within the cybersecurity domain [10, 37]. Beyond simple text classification, LLMs are now being explored for a wide range of complex cybersecurity tasks, including threat intelligence analysis [22, 29], malware code dissection [9, 13], penetration testing [7, 26], and incident response plan formulation [18]. Recognizing this potential, several cybersecurity and technology companies have begun developing their own security-specific LLMs. For instance, Microsoft has introduced Security Copilot [25], Google has developed Sec-PaLM as part of its Security AI Workbench [11], and CrowdStrike has launched Charlotte AI [6]. These models are typically trained on massive proprietary cybersecurity datasets and provide more accurate security analysis. However, their effectiveness in the unique task of alert triage remains largely unexamined. This gap underscores the urgency of systematically assessing LLMs to determine their practical utility in alleviating alert fatigue and enhancing SOC efficiency.

3 DATASET CONSTRUCTION

In this section, we introduce the construction process of the network alert dataset used in this evaluation. The pipeline consists of five major steps: alert collection, cleaning and normalization, sampling, anonymization, and expert annotation.

3.1 Dataset Scope and Design Principles

Our dataset focuses on **network-side security alerts**, i.e., alerts generated from network-facing security monitoring and detection systems such as NIDS, NIPS, WAF, and related traffic inspection platforms. More specifically, we include only alerts that exhibit clear semantic content and can be reasonably interpreted by a **Tier-1 security analyst** through manual inspection. In practice, such alerts typically contain semantically meaningful information, for example, rule names, request/response fields, payload snippets, protocol metadata, or other textual evidence, that allows an analyst to make an initial judgment about whether the event is suspicious, or benign. By contrast, alerts that are purely numerical, overly sparse, or heavily dependent on external correlation are outside the primary scope of this benchmark. If an alert cannot be reliably determined based on the evidence available in the alert itself, we do not force it into a definitive class.

3.2 Data Sources

The primary data source of our benchmark consists of network alert logs collected from the real-world production environments of three large-scale enterprises. These alerts were generated by multiple types of SOC-deployed security devices, including NIDS, NIPS, WAF, and other network-side monitoring sensors, and were centrally aggregated through a SIEM platform for downstream analysis and management. Our collection covers the period from December 2023 to May 2025 and contains over five million raw alert records. Although the logs exhibit minor temporal gaps caused by operational factors such as maintenance windows and temporary sensor outages, we verified that these gaps are limited in scope and do not materially compromise the dataset's size, diversity, or representativeness for evaluating LLMs on real-world network alert triage. An overview of the collected raw alert data is presented in Table 1.

Although the enterprise data constitute the primary source of our benchmark, relying exclusively on real-world production alerts would lead to a highly imbalanced dataset, because confirmed attack events are relatively rare in operational SOC environments compared with benign or false-positive alerts. To alleviate this issue and improve the coverage of attack behaviors, we additionally incorporated samples from two publicly available datasets: *AIT-ADS (AIT Alert Data Set)* [16] and *SABU Alert Dataset* [12]. These public resources were used as supplementary data sources to enrich attack-related samples and enhance the diversity of security scenarios represented in the benchmark.

3.3 Dataset Construction Pipeline

3.3.1 Data Cleaning and Normalization. To construct a high-quality benchmark from heterogeneous raw alerts, we first performed a unified data cleaning and normalization process over the collected logs from different enterprises and security devices. Specifically, we removed duplicated alerts, corrupted records, and entries with severely incomplete key fields. We then normalized alerts from different sources into a unified representation by aligning common fields such as alert name, device type, timestamp, source and destination information, protocol metadata, request or response content, and

Table 1: Overview of raw enterprise alert logs collected from production SOC environments.

Source	Sensor Type	Logs	Size (MB)	Period
Enterprise A	IDS/IPS (Type 1)	3,289,233	8,202.24	May 2023–May 2025
	WAF	53,010	57.40	Jun 2023–May 2025
	IDS/IPS (Type 2)	1,366,746	2,928.64	Jun 2023–May 2025
	IDS/IPS (Type 3)	258,460	150.00	Sep 2023–Jan 2025
Enterprise B	IDS	10,000	32.70	Feb 2025
Enterprise C	WAF	45,000	19.40	Feb 2025–May 2025
	IPS	7,063	4.12	Apr 2025–May 2025
Total	–	**5,029,512**	**11,394.50**	–

payload-related textual evidence. For records containing encoded or irregularly formatted content, we applied basic normalization procedures to improve readability and consistency across sources. Through this step, the original multi-source alert logs were transformed into a cleaner, more consistent, and semantically interpretable corpus.

3.3.2 Sampling Strategy. Because manually annotating all collected raw alerts was infeasible at the scale of several million records, we adopted a sampling strategy to construct a representative yet manageable benchmark for expert labeling. Our goal was not to mirror the raw frequency distribution of production alerts, which is often dominated by a small number of repetitive benign or low-value events, but rather to preserve the diversity of alert sources, device types, rule categories, and security scenarios that are relevant to Tier-1 triage. To this end, we employed a stratified sampling procedure over the cleaned alert corpus. Specifically, we first grouped alerts by enterprise source and security device type, and then further considered alert rule and time period within each group to improve coverage across heterogeneous operational settings. Within each stratum, we aimed to retain representative high-frequency patterns while also preserving lower-frequency but semantically important alerts, so that the resulting dataset would better capture the practical complexity of real-world analyst decision making. In addition, because confirmed attack events are relatively rare in production SOC environments, we supplemented the enterprise data with selected samples from public attack-oriented datasets to improve the coverage of adversarial behaviors and reduce the extreme class imbalance inherent in real-world deployments. Through this strategy, we constructed a dataset that balances practical representativeness, semantic diversity, and annotation feasibility, thereby making it suitable for systematic evaluation of LLMs in alert triage.

3.3.3 Labeling Scheme. In this study, we adopt a binary classification approach for labeling the alerts, focusing on the core decision-making task in Tier-1 alert triage: determining whether a given alert should be classified as an attack-related event or a non-attack event. This aligns with the operational goal of Tier-1 triage, which is to quickly filter out non-attack events and prioritize those that warrant further investigation. The two labels in our binary classification scheme are as follows:

- **Attack**: Alerts that are determined to indicate an actual security threat or malicious activity. These alerts exhibit strong evidence of an attack attempt or exploitation behavior with clear malicious intent that justifies escalation to Tier-2 analysts or further automated processing.
- **Non-Attack**: Alerts that do not represent genuine security threats. This category includes benign activities such as informational events that do not involve any attack behavior, or false positives triggered by legitimate business activities or normal operations that are misinterpreted by security systems.

Alerts that could not be reliably judged based on the evidence contained in the alert record itself were excluded from the final annotated benchmark. Such cases usually require additional information, such as asset context, historical behavior, business background, threat intelligence, or correlation with other logs, before a reliable decision can be made. Since our study focuses on the initial alert filtering task performed at Tier-1, these context dependent cases fall outside the scope of the benchmark.

3.3.4 Anonymization. Because the benchmark was constructed from production enterprise SOC data, privacy protection and compliance were treated as important requirements throughout the dataset construction process. Before annotation and evaluation, we applied targeted anonymization to fields that directly contained sensitive information. For example, in weak password alerts, plaintext passwords and other credential like values were replaced with placeholders. During anonymization, our objective was to protect directly exposed private information while preserving the evidence needed for security analysis. We therefore modified only sensitive values that were not necessary for label determination, while retaining the surrounding alert context, rule semantics, protocol information, and payload structure whenever possible. We also avoided excessive sanitization, since removing too much context could weaken the distinction between malicious activity and benign business traffic. All data handling procedures followed the data governance requirements of the participating enterprises, ensuring that private information was protected without reducing the usefulness of the benchmark for realistic Tier-1 alert triage evaluation.

3.3.5 Expert Annotation. The sampled alerts were annotated by a team of five security professionals, each with more than five years of practical SOC experience. This team included analysts and senior experts who had worked on alert triage, incident investigation, rule tuning, and network threat analysis in enterprise environments. Before formal annotation, all annotators were provided with a written guideline that defined the label taxonomy, decision criteria, and handling rules for ambiguous cases. A pilot calibration round was conducted on a shared subset of alerts to align annotators' understanding of suspicious behaviors, benign business traffic, and evidence insufficiency.

Each alert was labeled at the single-alert level using only the evidence realistically available during Tier-1 triage. Specifically, annotators examined the normalized alert record, including the rule name, device type, source and destination information, protocol metadata, url and query parameters, request headers, request body, response body, payload snippets, and decoded content. When necessary, annotators could consult public rule descriptions or vulnerability references to better interpret attack signatures and protocol semantics, but they were not allowed to use hidden investigation results or post hoc enterprise context unavailable at the time of initial triage.

Table 2: Basic distribution of the final annotated dataset.

Label	Alert Logs			Alert Types		
	Total	Count	Ratio	Total	Count	Top-5 Cov.
Attack	8322	2496	29.99%	241	210	46.23%
Non-Attack		5826	70.01%		62	67.68%

To improve annotation rigor, we adopted a dual independent annotation workflow followed by expert adjudication, a common practice in corpus construction and annotation quality control [3]. Each sampled alert was first independently labeled by two annotators, both of whom assigned a label according to the predefined guideline and the evidence contained in the alert record. If the two annotators reached the same decision, the label was directly accepted. If their labels disagreed, the sample was escalated to senior security experts for further review. During adjudication, the experts re-examined the alert evidence, discussed the source of disagreement, and assigned the final label according to the written guideline. This process helped reduce individual subjectivity and ensured that difficult cases were resolved through expert consensus rather than arbitrary judgment.

3.4 Dataset Statistics

Following the dataset construction pipeline, we built a real-world benchmark for Tier-1 network alert triage, comprising 8,322 annotated alerts with binary labels of *Attack* and *Non-Attack*. Table 2 summarizes the basic distribution of the final annotated dataset, including the number of alert logs, label ratio, alert types, and top-type coverage. The benchmark preserves the key evidence required for Tier-1 triage, such as rule names, protocol information, request and response content, and payload-related semantics. Due to space limitations, a processed alert example and additional dataset statistics are provided in Appendix C.

4 METHODOLOGY

In this section, we describe the methodology used to evaluate LLMs for Tier-1 network alert triage. We first formulate the alert triage task and introduce the models included in our evaluation. Beyond aggregate classification performance, we further design a set of multidimensional behavioral analyses to characterize how LLMs behave under realistic Tier-1 triage conditions. These analyses cover four aspects: response determinism, configuration sensitivity, reasoning correctness, and alert-type generalization.

4.1 Task Formulation

In this study, we formulate network alert triage as a single-alert classification task. Specifically, given one preprocessed alert instance as input, the model is required to assign it to one of predefined categories: *Attack* or *Non-Attack*. Each input alert contains only the information directly available in the alert record itself, such as the rule name, device type, protocol metadata, source and destination fields, and semantically meaningful request, response, or payload content. No additional external context is provided during inference, including cross-log correlation, historical alerts, asset context, business background knowledge, or analyst feedback.

Table 3: Overview of the selected LLMs evaluated in this study. CW denotes context window, KC denotes knowledge cutoff, and N/D denotes not disclosed. For MoE models, the size column reports total parameters with activated parameters in parentheses.

Model	Size	Developer	CW	KC
Proprietary Models				
GPT-5.1-Chat	N/D	OpenAI	128K	Sep. 2024
Gemini-3-Flash	N/D	Google	1M	Jan. 2025
Claude-Sonnet-4.5	N/D	Anthropic	200K	Jan. 2025
Open-Weight Models				
Llama-3.1-8B-Instruct	8B	Meta	128K	Dec. 2023
Llama-3.1-70B-Instruct	70B	Meta	128K	Dec. 2023
Llama-3.1-405B-Instruct	405B	Meta	128K	Dec. 2023
Qwen3-14B	14.8B	Alibaba	128K	N/D
Qwen3-32B	32.8B	Alibaba	128K	N/D
Qwen3-30B-A3B-Instruct-2507	30B (3B act.)	Alibaba	256K	N/D
Qwen3-235B-A22B-Instruct-2507	235B (22B act.)	Alibaba	256K	N/D
Kimi-K2-Instruct-0905	1T (32B act.)	Moonshot	256K	N/D
DeepSeek-V3.1	671B (37B act.)	DeepSeek	128K	N/D
GLM-4.7	355B (32B act.)	Z.ai	200K	N/D
Security-Finetuned Models				
Foundation-Sec-8B-Instruct	8B	Cisco	4K	N/D
SecGPT-14B	14B	CloudItera	128K	N/D
SecGPT-7B	7B	CloudItera	128K	N/D

4.2 Model Selection

To obtain a comprehensive view of LLM capabilities in network alert triage, we select a representative set of models for evaluation. Specifically, the evaluated models are organized into three groups: **Proprietary Models**, **Open-Weight Models**, and **Security-Finetuned Models**. This selection is intended to cover models with diverse parameter scales and varying degrees of domain specialization, thereby enabling a more systematic comparison of their effectiveness on the alert triage task. A complete list of the selected models is presented in Table 3.

4.3 Evaluation Dimensions and Metrics

To systematically evaluate LLMs for Tier-1 network alert triage, we assess model behavior from six complementary dimensions. We first evaluate overall effectiveness of LLMs in alert triage, which reflect the practical utility of LLMs in filtering SOC alerts. We then conduct four analyses to characterize the multidimensional behavior of LLMs, including response determinism, configuration sensitivity, reasoning accuracy and alert-type generalization. For each dimension, we define the corresponding experimental protocol and evaluation metrics.

4.3.1 Overall Effectiveness. We first evaluate the overall effectiveness of LLMs in Tier-1 alert triage. Since the task is formulated as a binary classification problem, we treat *Attack* as the positive class and *Non-Attack* as the negative class. In SOC operations, the most critical metrics are **True Positive Rate (TPR)** and **False Positive Rate (FPR)**, which measure the model's ability to detect real attacks and its tendency to incorrectly escalate benign alerts, respectively. They are defined as:

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN} \tag{1}$$

where TP and TN denote correctly classified attack and non-attack alerts, respectively, while FN denotes missed attacks and FP denotes non-attack alerts incorrectly classified as attacks. A practical Tier-1 triage model is expected to achieve a high TPR while maintaining a low FPR, so that it can reduce missed threats without introducing excessive alert noise.

In addition to TPR and FPR, we report standard classification metrics, including **Accuracy (Acc)**, **Precision (Pre)**, and **F1-score**, to provide a more comprehensive view of overall triage effectiveness. Model outputs that cannot be parsed into either *Attack* or *Non-Attack* are treated as incorrect predictions.

4.3.2 Response Determinism. Response determinism evaluates whether an LLM can produce stable triage decisions when given the same alert multiple times. This property is important for alert triage, since inconsistent decisions on the same alert may reduce analyst trust and make the model difficult to deploy in SOC workflows. We measure determinism using **Consistency Rate (CR)**, which records the proportion of alerts for which the model produces the same prediction across repeated runs, regardless of whether the prediction is correct:

$$CR = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(y_i^1 = y_i^2 = \cdots = y_i^K) \tag{2}$$

where N is the number of alerts, K is the number of repeated runs, and y_i^k denotes the prediction for the i-th alert in the k-th run.

4.3.3 Configuration Sensitivity. Configuration sensitivity evaluates how LLM alert triage performance changes under different experimental settings. This analysis helps us understand whether model behavior is robust or highly dependent on specific inference configurations. Specifically, we examine three types of configurations that may affect model behavior during alert triage:

- **Prompting strategies.** We compare five prompting strategies, including zero-shot prompting as the baseline, user-baseline prompting, one-shot prompting, few-shot prompting and chain-of-thought (CoT) prompting. These settings are used to examine how different levels of task instruction, demonstrations, and reasoning guidance affect alert triage performance. Detailed descriptions and prompt contents are provided in Appendix D.
- **Quantization levels.** For locally deployed open-weight models, we evaluate three precision settings: fp16 as the baseline, 8 bit quantization, and 4 bit quantization. This comparison is used to analyze whether lower precision deployment affects model effectiveness in alert triage.
- **Sampling parameters.** We further examine the impact of decoding randomness by varying two common sampling parameters: temperature and top_p. This allows us to evaluate whether model decisions remain stable under different generation settings.

For all configuration sensitivity experiments, we use the same effectiveness metrics as defined in § 4.3.1.

4.3.4 Reasoning Accuracy. Reasoning accuracy evaluates whether the final decision produced by an LLM is supported by a correct reasoning process. This analysis is conducted under CoT prompting strategies, where the model is required to provide an explicit reasoning process before outputting the final label.

We categorize each output into four cases:

- **Correct decision with correct reasoning.** The model gives the correct label and its reasoning is consistent with the evidence in the alert. This case is considered reliable.
- **Correct decision with incorrect reasoning.** The model gives the correct label but its reasoning is flawed or unsupported, indicating that the decision may be correct by chance.
- **Incorrect decision with incorrect reasoning.** Both the final label and the reasoning process are incorrect, making the output unreliable.
- **Incorrect decision with correct reasoning.** The reasoning process identifies relevant evidence, but the final decision contradicts or fails to follow the reasoning. This case is also considered unreliable.

We use **Correct Reasoning Rate (CRR)** to measure the proportion of outputs that have both correct decisions and correct reasoning:

$$CRR = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(\hat{y}_i = y_i \wedge r_i = 1) \tag{3}$$

where N is the number of evaluated alerts, $\hat{y}_i$ is the model prediction, y_i is the ground-truth label, and $r_i = 1$ indicates that the reasoning process is correct and consistent with the alert evidence.

4.3.5 Alert-Type Generalization. Alert-type generalization evaluates whether the triage capability of LLMs remains stable across different categories of alerts. Since real-world SOC alerts are highly diverse and contain a large number of fine-grained alert types, directly evaluating each individual type may lead to overly fragmented results. Therefore, we first merge alert types with similar rule semantics, detection logic, or attack characteristics into broader alert categories, and then evaluate model performance within each category. For example, alerts triggered by command injection rules and remote code execution rules are both grouped into the *Code Execution* category, because they reflect similar execution-oriented attack behaviors. For each alert category, we use the same effectiveness metrics defined in § 4.3.1.

5 EVALUATION

In this section, we present the empirical evaluation of LLMs on SecAlertBench following the multidimensional analyses described in § 4. Our goal is to systematically assess whether current LLMs can serve as effective Tier-1 alert triage assistants in real-world SOC scenarios. To guide this evaluation, we organize our analysis around the following three research questions:

- **RQ1**: How effective are LLMs for Tier-1 alert triage in SOCs?
- **RQ2**: What multidimensional characteristics do LLMs exhibit in Tier-1 alert triage?
- **RQ3**: What are the main limitations and bottlenecks of LLMs in Tier-1 alert triage?

5.1 Overall Effectiveness of LLMs in Alert Triage

5.1.1 Experiment setup. To answer RQ1, we evaluate all selected LLMs under a unified single-alert triage setting. Each model is given one preprocessed alert record as input and is required to output a classification decision based only on the information contained in

Table 4: Overall performance of the evaluated LLMs on RQ1. All values are reported in %. indicates the best result for each metric, and indicates the worst result.

Model[🔔]	Acc.↑	Prec.↑	TPR↑	FPR↓	F1↑
GPT-5.1-Chat	74.90	67.32	96.80	47.00	79.41
Gemini-3-Flash	85.65	79.10	96.90	25.60	87.10
Claude-Sonnet-4.5	64.65	58.86	97.30	68.00	73.35
Llama-3.1-8B-Instruct	42.30	44.70	65.00	80.40	52.97
Llama-3.1-70B-Instruct	79.10	78.98	79.30	21.10	79.14
Llama-3.1-405B-Instruct	67.85	64.00	81.60	45.90	71.74
Qwen3-14B	62.35	60.82	69.40	44.70	64.83
Qwen3-32B	60.50	59.07	68.40	47.40	63.39
Qwen3-30B-A3B-Instruct	62.30	59.70	75.70	51.10	66.75
Qwen3-235B-A22B-Instruct	68.55	62.49	92.80	55.70	74.69
Kimi-K2-Instruct	86.90	86.61	87.30	13.50	86.95
DeepSeek-V3.1	73.95	72.11	78.10	30.20	74.99
GLM-4.7	73.95	69.78	84.50	36.60	76.44
Foundation-Sec-8B-Instruct	64.15	85.46	34.10	5.80	48.75
SecGPT-14B	57.75	55.21	82.10	66.60	66.02
SecGPT-7B	59.80	56.43	86.00	66.40	68.15
Avg	**67.79**	**66.29**	**79.71**	**44.13**	**70.92**

🔔 For brevity, we use shortened model names in subsequent tables and the main text. For example, Qwen3-30B-A3B-Instruct-2507 is abbreviated as Qwen3-30B-A3B-Instruct. The exact model versions are listed in Table 3.

that alert, without access to external threat intelligence, historical alerts, asset context, or cross-log correlation. For each experiment, we randomly sample 1,000 alerts from the Attack class and 1,000 alerts from the Non-Attack class, resulting in a balanced test set of 2,000 alerts. This balanced setting allows us to compare model behavior under equal class priors and analyze the trade-off between attack detection and false-positive control. For proprietary models and large-scale open-weight models that are not feasible to fully deploy locally, we use the official APIs provided by their developers or service platforms. For open-weight models that can be locally deployed, we conduct full local inference using their released checkpoints. To ensure a fair comparison, all models are evaluated with the same zero-shot prompt template, and the temperature is set to 0 to minimize decoding randomness. We compare model predictions against expert-annotated labels and report Accuracy, Precision, TPR, FPR, and F1-score.

5.1.2 Effectiveness Analysis. The effectiveness results of all evaluated LLMs are summarized in Table 4. Based on these results, we make the following observations.

Overall effectiveness and leading models. Across all evaluated models, the average Accuracy, Precision, TPR, FPR, and F1-score are 67.79%, 66.29%, 79.71%, 44.13%, and 70.92%, respectively. The relatively high average TPR shows that current LLMs can identify a substantial portion of attack-related alerts, suggesting their potential as Tier-1 triage assistants. However, the average FPR of 44.13% indicates that many benign alerts are still incorrectly escalated as attacks, which remains a major obstacle for practical SOC deployment. Among individual models, Gemini-3-Flash and Kimi-K2-Instruct achieve the strongest overall performance. Gemini-3-Flash obtains the highest F1-score of 87.10% and a very high TPR of 96.90%, while Kimi-K2-Instruct achieves the highest Accuracy of 86.90%, the highest Precision of 86.61%, and a low FPR of 13.50%. These results suggest that strong triage performance requires not only detecting attacks, but also effectively suppressing false positives.

Table 5: Prediction distributions of LLMs on Attack, Non-Attack, and overall samples. denotes predictions as Attack, and denotes predictions as Non-Attack.

Model	Attack	Non-Attack	Overall
GPT-5.1-Chat	968/32	470/530	1438/562
Gemini-3-Flash	969/31	256/744	1225/775
Claude-Sonnet-4.5	973/27	680/320	1653/347
Llama-3.1-8B-Instruct	650/350	804/196	1454/546
Llama-3.1-70B-Instruct	793/207	211/789	1004/996
Llama-3.1-405B-Instruct	816/184	459/541	1275/725
Qwen3-14B	694/306	447/553	1141/859
Qwen3-32B	684/316	474/526	1158/842
Qwen3-30B-A3B-Instruct	757/243	511/489	1268/732
Qwen3-235B-A22B-Instruct	928/72	557/443	1485/515
Kimi-K2-Instruct	873/127	135/865	1008/992
DeepSeek-V3.1	781/219	302/698	1083/917
GLM-4.7	845/155	366/634	1211/789
Foundation-Sec-8B-Instruct	341/659	58/942	399/1601
SecGPT-14B	821/179	666/334	1487/513
SecGPT-7B	860/140	664/336	1524/476

Trade-off between attack detection and false-positive control. A key observation is that high TPR alone does not guarantee effective alert triage. As shown in Table 5, most LLMs can correctly identify a large fraction of Attack samples, with many models classifying more than 800 out of 1,000 Attack alerts as Attack. However, this strong attack-side detection is often accompanied by an aggressive prediction bias toward the Attack class. For example, Claude-Sonnet-4.5 correctly classifies 973 Attack samples as Attack, but also misclassifies 680 out of 1,000 Non-Attack samples as Attack; similarly, GPT-5.1-Chat detects 968 Attack samples, while still escalating 470 Non-Attack samples. These results indicate that current LLMs are generally sensitive to attack-like evidence, but they often struggle to suppress false positives, which can significantly increase the burden on downstream analysts.

Differences across model categories. The three model groups exhibit different behavior patterns. Proprietary models generally achieve strong attack detection capability, but their false-positive rates are relatively high, possibly because some models adopt stricter safety-alignment strategies and therefore tend to classify suspicious or ambiguous alerts as attacks. Open-weight models show larger performance variance: smaller models such as Llama-3.1-8B-Instruct perform poorly, whereas stronger models such as Kimi-K2-Instruct, Llama-3.1-70B-Instruct, DeepSeek-V3.1, and GLM-4.7 achieve competitive results. Security-finetuned models do not consistently outperform general-purpose LLMs. Foundation-Sec-8B-Instruct achieves the lowest FPR of 5.80%, but its TPR is only 34.10%, indicating an overly conservative tendency. Conversely, SecGPT-14B and SecGPT-7B achieve higher TPRs but suffer from high FPRs above 66%. This suggests that security-domain fine-tuning alone is insufficient for Tier-1 alert triage.

Table 6: Response determinism of selected LLMs under different temperature settings. indicates the best result within each model group, and indicates the worst result.

Model	Temp.	Consistent / Total	CR ↑	Avg. Agree. ↑
Kimi-K2-Instruct	0.0	463 / 500	92.6	9.838 / 10
	0.5	401 / 500	80.2	9.586 / 10
	1.0	365 / 500	73.0	9.388 / 10
	1.5	203 / 500	40.6	8.780 / 10
Qwen3-30B -A3B-Instruct	0.0	408 / 500	81.6	9.702 / 10
	0.5	315 / 500	63.0	9.388 / 10
	1.0	288 / 500	57.6	9.304 / 10
	1.5	302 / 500	60.4	9.282 / 10

Finding 1: Current LLMs show promising effectiveness in Tier-1 alert triage, achieving a high average TPR of 79.71% and identifying a large fraction of attack alerts. However, their high average FPR of 44.13% reveals a strong tendency to over-escalate benign alerts.

5.2 Multidimensional Behavioral Analysis of LLMs in Alert Triage

5.2.1 Response Determinism. To evaluate response determinism, we construct a fixed subset of $N = 500$ alerts, including 250 Attack and 250 Non-Attack samples. Following Equation 2, each alert is independently queried $K = 10$ times using the same baseline prompt, without conversation history, external context, or feedback from previous outputs. To examine the impact of decoding randomness, we repeat the experiment under four temperature settings, i.e., temperature $\in \{0, 0.5, 1.0, 1.5\}$, while keeping all other inference parameters unchanged. Considering the high cost of repeated inference, we adopt a deployment-oriented representative selection strategy rather than evaluating all models. Specifically, we select Kimi-K2-Instruct and Qwen3-30B-A3B-Instruct for response determinism analysis. Kimi-K2-Instruct is queried through an API service and achieves strong overall performance in RQ1, making it a practical representative of API-based deployment. Qwen3-30B-A3B-Instruct is an open-weight model deployed locally, allowing us to control the inference framework and decoding parameters, and thus serves as a representative of self-hosted deployment.

Observations. As shown in Table 6, both models achieve the highest response determinism under the lowest-temperature setting. For Kimi-K2-Instruct, the CR decreases substantially from 92.6% at temperature 0.0 to 40.6% at temperature 1.5. Qwen3-30B-A3B-Instruct shows a similar trend, with the highest CR of 81.6% at temperature 0.0 and noticeably lower consistency under higher-temperature settings. Although the average agreement remains relatively high in most cases, the strict consistency rate drops as decoding randomness increases, indicating that even a small number of deviating outputs can reduce the reliability of repeated alert triage decisions. These results suggest that, in noise-reduction-oriented SOC alert triage, where stable and reproducible filtering decisions are more important than output diversity, LLMs should preferably be deployed with a low temperature, ideally temperature 0, to obtain more deterministic results.

Finding 2: LLM-based alert triage becomes less deterministic as decoding randomness increases, suggesting that low-temperature inference, especially temperature 0, is preferable for stable and reproducible SOC alert triage.

5.2.2 Configuration Sensitivity. To analyze how experimental configurations affect LLM-based alert triage, we conduct a controlled sensitivity study based on the configuration settings defined in § 4.3.3. Starting from the baseline evaluation setup, we vary only one configuration factor at a time while keeping the label parser, input format, and other unrelated inference settings unchanged. For each experimental round, we randomly sample a balanced subset of $N = 500$ alerts, including 250 Attack and 250 Non-Attack samples. Considering the cost of evaluating all models under all configurations, and the fact that different configuration factors apply to different deployment scenarios, we select representative models separately for each analysis. For prompting strategies, we evaluate GPT-5.1-Chat, Kimi-K2-Instruct, Qwen3-30B-A3B-Instruct, and Foundation-Sec-8B-Instruct to cover proprietary, large-scale open-weight, locally deployable open-weight, and security-finetuned models. For quantization levels, we focus on Qwen3-32B and Qwen3-30B-A3B-Instruct, since quantization is mainly applicable to locally deployed open-weight models. For sampling parameters, we select Kimi-K2-Instruct and Qwen3-30B-A3B-Instruct as representative models for API-based and self-hosted deployment settings, respectively.

Impact of Prompting Strategies. Table 7 shows that prompting strategies have a substantial impact on LLM-based alert triage, especially in controlling false positives. Overall, few-shot prompting achieves the best balance between attack detection and false-positive reduction across the evaluated models. For GPT-5.1-Chat, although the zero-shot baseline obtains the highest TPR of 96.40%, it also produces the worst FPR of 48.80%, whereas few-shot prompting reduces the FPR to 9.60% and achieves the best F1-score of 91.70%. A similar pattern is observed for Kimi-K2-Instruct and Qwen3-30B-A3B-Instruct, where few-shot prompting achieves the best F1-scores of 89.57% and 83.15%, respectively, while also yielding the lowest FPRs. This suggests that providing task-relevant demonstrations can help models better calibrate their decision boundaries and avoid overly aggressive attack predictions. In contrast, CoT prompting does not consistently improve performance; for example, GPT-5.1-Chat with CoT still suffers from a high FPR of 47.20%, close to the zero-shot baseline. The user-baseline prompt often makes models more conservative, reducing FPR in some cases but substantially hurting TPR and F1, as seen in Foundation-Sec-8B-Instruct, where the FPR drops to 0.40% but the TPR falls to only 19.20%. These results indicate that prompt design mainly affects the trade-off between detection sensitivity and false-positive control, with few-shot prompting providing the most robust overall improvement.

Impact of Quantization Levels. As shown in Table 8, quantization has a noticeable but relatively limited impact compared with prompting strategies. For both Qwen3-32B and Qwen3-30B-A3B-Instruct, the fp16 setting generally achieves the best overall balance in Accuracy, Precision, FPR, and F1-score. Although lower-bit quantization sometimes increases TPR, especially under the 4bit setting, it also leads to a clear increase in FPR, indicating weaker false-positive

Table 7: Impact of prompting strategies on selected representative LLMs. All values are reported in %. indicates the best result in each metric column, and indicates the worst result.

Prompting Strategy	GPT-5.1-Chat			Kimi-K2-Instruct			Qwen3-30B-A3B-Instruct			Foundation-Sec-8B-Instruct		
	TPR↑	FPR↓	F1↑	TPR↑	FPR↓	F1↑	TPR↑	FPR↓	F1↑	TPR↑	FPR↓	F1↑
Zero-shot	96.40	48.80	78.63	84.80	15.60	84.63	79.20	56.40	67.23	30.00	6.00	44.12
User-baseline	85.20	33.20	78.02	74.80	9.20	81.30	82.40	50.80	70.67	19.20	0.40	32.11
One-shot	90.00	22.40	84.75	87.20	11.20	87.90	86.80	47.20	74.19	75.60	32.80	72.55
Few-shot	92.80	9.60	91.70	87.60	8.00	89.57	90.80	27.60	83.15	72.00	15.60	76.76
CoT	96.00	47.20	78.95	86.40	10.00	87.98	82.40	46.00	72.15	52.80	12.00	64.08

Table 8: Impact of quantization levels on representative open-weight LLMs. All values are reported in %. indicates the best result within each model group, and indicates the worst result.

Model	Quant.	Acc.↑	Prec.↑	TPR↑	FPR↓	F1↑
Qwen3-32B	fp16	63.20	59.82	80.40	54.00	68.60
	8bit	62.40	59.17	80.00	55.20	68.03
	4bit	57.40	54.81	84.40	69.60	66.46
Qwen3-30B -A3B-Instruct	fp16	63.20	60.65	75.20	48.80	67.14
	8bit	60.00	56.98	81.60	61.60	67.11
	4bit	58.60	55.89	81.60	64.40	66.34

control. Overall, these results suggest that quantization mainly affects the trade-off, but its impact is less dominant than prompt design.

Impact of Sampling Parameters. As shown in Figure 2, sampling parameters also have a relatively limited impact on alert triage performance compared with prompting strategies. For Kimi-K2-Instruct, TPR remains consistently high across all configurations, ranging from 86.0% to 89.2%, while FPR varies from 8.0% to 15.2%. Qwen3-30B-A3B-Instruct shows an even more stable TPR, ranging only from 74.4% to 76.0%, although its FPR remains much higher, between 44.8% and 52.4%. We do not observe a clear monotonic trend with either temperature or top_p, suggesting that decoding randomness only marginally affects this deterministic classification task, while model-specific behavior remains the dominant factor.

Finding 3: LLM-based alert triage is most sensitive to prompt design, with few-shot prompting consistently improving the balance between attack detection and false-positive control. In contrast, quantization and sampling parameters have smaller impacts.

5.2.3 Reasoning Correctness. To evaluate reasoning correctness, we reuse the model outputs generated with the CoT prompt in § 4.3.3. For each model output, we first compare the final parsed label with the ground-truth label to determine whether the decision is correct. We then examine whether the generated reasoning is supported by the evidence in the alert record and whether it logically justifies the final decision. Each output is assigned to one of four cases: correct decision with correct reasoning, correct decision with incorrect reasoning, incorrect decision with incorrect reasoning, and incorrect decision with correct reasoning. Unparsable outputs or reasoning that is unsupported, contradictory, or irrelevant to the alert evidence

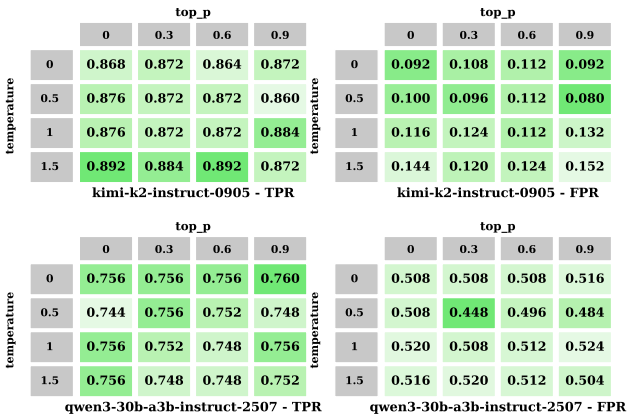

Figure 2: Impact of sampling parameters on alert triage performance. Each heatmap reports TPR or FPR under different temperature and top_p settings.

are treated as incorrect. Finally, we count the number of samples in each case for every model and compute the CRR according to Equation 3.

Observations. Table 9 shows that correct final decisions do not always imply reliable reasoning. GPT-5.1-Chat achieves the highest CRR of 64.60%, followed closely by Kimi-K2-Instruct with 63.20%, indicating that these two models more frequently produce decisions supported by valid alert evidence. However, their error patterns differ: Kimi-K2-Instruct produces many correct decisions with incorrect reasoning, suggesting that it may sometimes reach the right label without a faithful rationale, while GPT-5.1-Chat has more cases with correct reasoning but incorrect final decisions, revealing occasional reasoning-decision misalignment. Qwen3-30B-A3B-Instruct achieves a moderate CRR of 55.40%, whereas Foundation-Sec-8B-Instruct performs worst with only 40.60% CRR and a large number of incorrect or unsupported reasoning cases. Overall, these results suggest that CoT prompting can expose important reliability differences among models, and that evaluating only final classification accuracy may overestimate their trustworthiness in SOC alert triage.

Table 9: Reasoning correctness analysis of selected LLMs. CRR denotes the percentage of samples with both correct decisions and correct reasoning. In the header legend, D denotes the final decision, R denotes reasoning, and +/− denote correct/incorrect.

Model	CRR	Details ($D+/R+$ $D+/R-$ $D-/R+$ $D-/R-$)
Foundation-Sec -8B-Instruct	40.60%	203 149 46 102 /500
GPT-5.1-Chat	64.60%	323 49 116 12 /500
Kimi-K2-Instruct	63.20%	316 125 49 10 /500
Qwen3-30B -A3B-Instruct	55.40%	277 64 96 63 /500

Finding 4: Correct triage decisions do not always come with faithful reasoning. Even when the final label is correct, the decision may still be derived from flawed or unsupported reasoning.

5.2.4 Alert-Type Generalization. We further examine whether LLMs exhibit consistent triage capability across different categories of alerts. We reuse the prediction results obtained in the overall effectiveness evaluation described in § 5.1. We conduct this analysis based on the alert category mapping defined earlier and compute model performance within each category. This category-level analysis allows us to examine whether model effectiveness remains stable across different kinds of alert semantics, rather than being concentrated on a small number of specific alert rules. For model comparison, we include three proprietary models, namely GPT-5.1-Chat, Gemini-3-Flash, and Claude-Sonnet-4.5, as well as four competitive open weight models, namely Kimi-K2-Instruct, Qwen3-235B-A22B-Instruct, DeepSeek-V3.1, and GLM-4.7.

Observations. As shown in Table 10, LLM performance differs substantially across alert categories. *Directory Traversal* is the most stable category, where most models achieve nearly 100% TPR and F1 scores above 97%, indicating that alerts with explicit payload semantics are relatively easy for LLMs to recognize. In contrast, *Code Execution* and *SQL Injection* show strong attack detection but weak false positive control. For *Code Execution*, all models achieve around 99% to 100% TPR, but their FPR remains high. For *SQL Injection*, all models reach 100% TPR, yet most F1 scores are low because benign SQL like requests are frequently classified as attacks. *Sensitive Information Leakage* and *Unauthorized Access / Bypass* show larger model differences. Gemini-3-Flash performs consistently well in both categories, while several open weight models become overly conservative and miss many attacks. Overall, these results suggest that LLMs are effective at recognizing clear attack patterns, but their triage behavior remains sensitive to alert semantics and becomes less reliable when benign and malicious evidence are difficult to distinguish.

Table 10: Category-level performance of representative LLMs on selected alert categories. All values are reported in %. For each alert category and each metric, marks the best result among the compared models, while marks the worst result among the compared models.

Alert Type	Model	TPR↑	FPR↓	F1↑
Code Execution	GPT-5.1-Chat	100.00	55.32	94.09
	Gemini-3-Flash	100.00	72.00	91.43
	Claude-Sonnet-4.5	100.00	98.18	87.26
	Kimi-K2-Instruct	99.52	75.00	91.15
	Qwen3-235B-A22B-Instruct	100.00	92.16	89.44
	DeepSeek-V3.1	98.98	81.58	92.20
	GLM-4.7	100.00	71.43	92.31
Sensitive Info. Leakage	GPT-5.1-Chat	90.12	18.60	92.54
	Gemini-3-Flash	88.40	7.89	93.02
	Claude-Sonnet-4.5	90.76	29.03	92.78
	Kimi-K2-Instruct	60.56	6.67	74.66
	Qwen3-235B-A22B-Instruct	77.01	16.67	85.46
	DeepSeek-V3.1	49.21	8.82	65.28
	GLM-4.7	51.37	15.79	66.43
Directory Traversal	GPT-5.1-Chat	100.00	32.00	97.91
	Gemini-3-Flash	100.00	6.67	99.49
	Claude-Sonnet-4.5	100.00	19.23	98.74
	Kimi-K2-Instruct	100.00	28.00	98.06
	Qwen3-235B-A22B-Instruct	100.00	27.27	98.47
	DeepSeek-V3.1	97.81	24.00	97.28
	GLM-4.7	100.00	19.05	98.90
SQL Injection	GPT-5.1-Chat	100.00	54.87	12.44
	Gemini-3-Flash	100.00	24.65	24.15
	Claude-Sonnet-4.5	100.00	69.50	9.60
	Kimi-K2-Instruct	100.00	4.88	50.00
	Qwen3-235B-A22B-Instruct	100.00	60.88	9.20
	DeepSeek-V3.1	100.00	33.60	12.40
	GLM-4.7	100.00	41.11	13.38
Unauthorized Access / Bypass	GPT-5.1-Chat	80.77	76.92	73.68
	Gemini-3-Flash	84.62	33.33	83.02
	Claude-Sonnet-4.5	80.00	62.50	72.73
	Kimi-K2-Instruct	17.39	0.00	29.63
	Qwen3-235B-A22B-Instruct	33.33	43.75	40.00
	DeepSeek-V3.1	6.67	0.00	12.50
	GLM-4.7	24.00	25.00	35.29

Finding 5: LLM triage performance varies substantially across alert types, showing that strong overall performance does not imply stable generalization across alert categories. Alerts with clear payload semantics are easier to triage, while semantically ambiguous types often lead to excessive false positives or missed attacks.

5.3 Main Limitations and Bottlenecks

Based on the results in § 5.1 and § 5.2, current LLMs show promising capability in identifying attack-related alerts, but they are still not ready for direct deployment in real SOC Tier 1 workflows. To answer RQ3, we analyze their main bottlenecks from two perspectives: efficiency and performance. For efficiency, we measure the inference latency of representative LLMs and estimate their daily processing capacity, since Tier 1 triage must handle large volumes of alerts under strict time constraints. For performance, we focus on the most

Table 11: Efficiency evaluation under local deployment. Capacity per day is estimated as 86,400/Avg. Lat.. indicates the best result in each metric column, and indicates the worst result.

Model	Prompt	Avg. Lat. (s)↓	P95 Lat. (s)↓	Capacity/day↑	Peak Mem. (MB)↓
Qwen3-14B	Zero-shot	0.42	0.67	207,616	29,569
Qwen3-14B	Few-shot	1.18	1.66	73,159	31,072
Qwen3-14B	CoT	4.56	5.65	18,966	29,587
Qwen3-30B-A3B-Instruct	Zero-shot	0.27	0.38	320,065	59,362
Qwen3-30B-A3B-Instruct	Few-shot	0.65	0.90	133,311	60,553
Qwen3-30B-A3B-Instruct	CoT	7.21	8.72	11,989	59,371
Qwen3-32B	Zero-shot	0.92	1.52	93,639	64,551
Qwen3-32B	Few-shot	2.76	3.88	31,324	66,770
Qwen3-32B	CoT	10.06	12.66	8,592	64,583

prominent failure mode observed in our evaluation: excessive false positives. Through case studies of benign alerts that are incorrectly escalated as attacks, we investigate why LLMs tend to classify non-attack alerts as attacks.

5.3.1 Efficiency bottleneck: limited throughput for production SOCs. In production SOCs, alerts arrive continuously and often in bursts, so a deployable Tier 1 triage system must satisfy both accuracy and throughput requirements. Unlike rule matching or lightweight classifiers, LLM triage requires prompt construction, model inference, response generation, and decision parsing for each alert, which introduces substantial computational overhead. To quantify this bottleneck under a practical self-hosted setting, we conduct efficiency experiments on a single NVIDIA A100 GPU using FP16 precision, while keeping the inference framework, alert format, output parser, and decoding configuration consistent across models. We select three locally deployable and relatively strong open weight models, namely Qwen3-14B, Qwen3-32B, and Qwen3-30B-A3B-Instruct. For each model, we evaluate zero-shot, few-shot, and CoT prompts, corresponding to the baseline setting, the prompt setting that improves false positive control, and the reasoning-oriented setting.

Observations. The efficiency results are summarized in Table 11. Even under continuous local inference on a single A100 GPU, current LLMs cannot support million-scale Tier-1 alert triage as standalone engines. In the best zero-shot setting, Qwen3-30B-A3B-Instruct processes about 320K alerts per day, while Qwen3-14B and Qwen3-32B process about 208K and 94K alerts per day, respectively. This is still below the million-level daily alert volume observed in large SOCs. The gap becomes larger with longer prompts: few-shot prompting reduces capacity to 73K–133K alerts per day, and CoT prompting further drops it to only 8.6K–19.0K alerts per day. Therefore, processing one million alerts per day would require multiple A100 GPUs even under ideal linear scaling, and the actual deployment cost may be higher due to bursty traffic, queueing delay, and system overhead.

> **Finding 6:** Current LLMs still lack sufficient inference throughput to serve as standalone Tier-1 alert triage engines for million-scale daily SOC workloads.

5.3.2 Effectiveness bottleneck: excessive false positive escalation. Beyond efficiency, the most prominent effectiveness bottleneck of current LLMs is excessive false positive escalation. As shown in § 5.1, many models achieve relatively high TPRs, but this often comes with high FPRs, where benign alerts are incorrectly classified as attacks and escalated for further investigation. This is particularly problematic for Tier-1 alert triage, whose goal is not only to detect real threats, but also to reduce noisy alerts before they reach downstream analysts. To better understand this failure mode, we inspect representative false positive cases from the evaluated outputs, focusing on alerts whose ground-truth label is Non-Attack but whose model prediction is Attack. For each case, we examine the alert name, protocol metadata, request and response fields, payload content, and model output when available. Based on this analysis, we summarize the main causes of false positive escalation as follows.

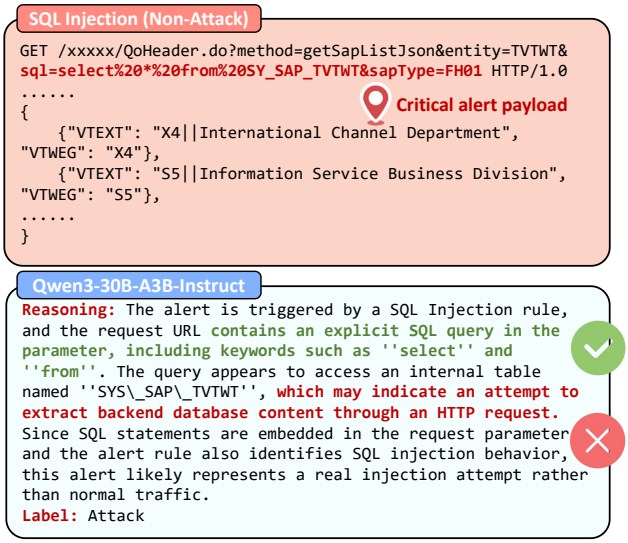

Figure 3: A representative false positive case caused by attack-like business traffic.

Failure pattern 1: Lack of operational context for attack-like business traffic. A major cause of false positive escalation is that some benign business activities exhibit surface patterns similar to real attacks, but their benign nature can only be determined with enterprise-specific operational context. As shown in Figure 3, the alert is triggered by an SQL Injection rule, and the request parameter contains an SQL-like expression that queries an internal enterprise table. From the local alert evidence alone, such tokens can easily be interpreted as an injection attempt, especially because the rule name and the payload both contain strong attack-related cues. However, this traffic may correspond to a legitimate business query generated by an internal application, and the SQL-like content may be exposed due to implementation details, legacy software behavior, or version-specific request formatting. Since the LLM only observes the single alert record and lacks contextual information such as the business purpose of the interface, the role of the source and destination assets, historical normal behavior, software version, and internal whitelist status, it may treat the attack-like business payload as malicious evidence and incorrectly classify the alert as Attack. This pattern shows that LLMs often struggle to distinguish benign business logic from true attacks when the same textual or structural features can appear in both.

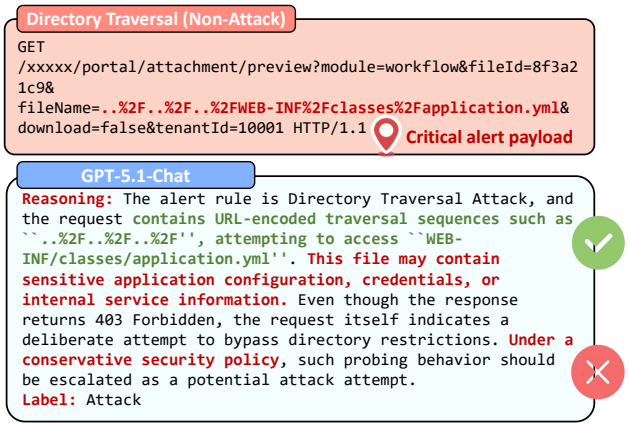

Figure 4: A representative false positive case caused by overemphasis on rule names and attack keywords.

Failure pattern 2: Overemphasis on rule names and attack keywords. Another source of false positive escalation is that LLMs may place excessive weight on salient attack-related cues, such as rule names and suspicious payload tokens. As shown in Figure 4, the alert is labeled as Directory Traversal Attack, and the request contains URL-encoded traversal sequences such as *..%2F..%2F..%2F*, together with a sensitive-looking path, *WEB-INF/classes/application.yml*. These cues strongly match known directory traversal attempts and can easily drive the model toward a conservative Attack decision. However, the response returns *403 Forbidden* and indicates that the file path is invalid, meaning that no internal file content is actually exposed. When the model mainly focuses on the rule name and attack-like tokens while underweighting counter-evidence such as blocked access, failed responses, or absence of data exposure, it may incorrectly escalate benign or blocked events and increase the false positive rate.

> **Finding 7:** High false positive rates arise from the combined effect of missing operational context, semantic ambiguity, and LLMs' security-sensitive decision behavior. Without enterprise-specific context, LLMs struggle to distinguish benign business traffic from real attacks when both share similar alert patterns. As a result, they often conservatively escalate ambiguous non-attack alerts as attacks.

6 DISCUSSION

6.1 Limitations

This study has two main limitations. First, our evaluation focuses on Tier-1 alert triage, which represents only the initial filtering and escalation stage of a broader SOC alert-handling workflow. In real-world operations, alerts are typically processed through multiple stages, where Tier-2 and Tier-3 analysts further correlate alerts with historical logs, asset information, threat intelligence, and incident context before making final investigation or response decisions. Therefore, our results mainly reflect the capability of LLMs to support early-stage triage, rather than their ability to handle the full alert investigation lifecycle. Second, we formulate the task as single-alert binary classification, requiring models to decide whether an individual alert is *Attack* or *Non-Attack* based only on the evidence contained in that alert. While this controlled setting enables systematic evaluation, it simplifies practical SOC decision-making, where many alerts can only be reliably judged through cross-alert correlation, business context, and richer operational evidence. Future work should extend the evaluation to multi-stage, context-aware SOC workflows and study whether LLMs can support more realistic incident-level reasoning and analyst decision-making.

6.2 Implications

LLMs as Assistive Components Rather than Standalone Engines. Our empirical evaluation highlights that current LLMs lack the inference throughput necessary to independently process the million-scale daily alert volumes typical of production SOCs. Furthermore, their tendency to aggressively escalate benign alerts results in high false-positive rates, which directly undermines the noise-reduction objective of Tier-1 triage. Consequently, rather than deploying LLMs as standalone automated decision engines, industry practitioners should position them as assistive components. In a human-in-the-loop workflow, LLMs can significantly accelerate manual triage by summarizing complex payloads, extracting semantic features, and offering preliminary risk assessments, leaving the final escalation decisions to human analysts.

Necessity of Enterprise Context and Alert Correlation. The observed failure patterns reveal that LLMs frequently struggle to differentiate genuine attacks from benign, attack-like business traffic when relying solely on isolated alert records. This semantic ambiguity demonstrates that text-based reasoning is insufficient without external operational knowledge. To achieve reliable real-world deployment, future SOC architectures must move beyond single-prompt inference and integrate LLMs with comprehensive enterprise context. By anchoring model decisions to asset databases, cross-log correlation, historical behavioral baselines, and up-to-date threat intelligence, organizations can effectively mitigate false-positive escalations and bridge the gap between abstract model reasoning and complex operational realities.

7 CONCLUSION

In this paper, we introduced SecAlertBench, a comprehensive, real-world benchmark comprising 8,322 expert-annotated logs across 241 alert types, designed to systematically evaluate LLMs for Tier-1 network alert triage. Through an extensive evaluation of 16 representative LLMs, we demonstrated that while current models exhibit promising attack-detection capabilities, they consistently struggle with high false-positive rates. Our multidimensional analysis reveals that this performance gap is largely driven by a lack of enterprise-specific operational context, semantic ambiguity between benign and malicious traffic, and an inherent over-sensitivity to attack-related keywords. Furthermore, practical deployment is currently hindered by inconsistent reasoning and insufficient inference throughput for million-scale daily SOC workloads. Ultimately, our findings conclude that LLMs are not yet ready to serve as standalone automated triage engines. Instead, they are best suited as assistive tools in human-in-the-loop workflows, augmented by cross-log correlation, threat intelligence, and operational context.

REFERENCES

[1] Bushra A Alahmadi, Louise Axon, and Ivan Martinovic. 2022. 99% false positives: A qualitative study of {SOC} analysts' perspectives on security alarms. In *31st USENIX Security Symposium (USENIX Security 22)*. 2783–2800.

[2] Muhamad Erza Aminanto, Lei Zhu, Tao Ban, Ryoichi Isawa, Takeshi Takahashi, and Daisuke Inoue. 2019. Combating threat-alert fatigue with online anomaly detection using isolation forest. In *International Conference on Neural Information Processing*. Springer, 756–765.

[3] Ron Artstein and Massimo Poesio. 2008. Inter-Coder Agreement for Computational Linguistics. *Computational Linguistics* 34, 4 (12 2008), 555–596. https://doi.org/10.1162/coli.07-034-R2 arXiv:https://direct.mit.edu/coli/article-pdf/34/4/555/1808947/coli.07-034-r2.pdf

[4] Tao Ban, Ndichu Samuel, Takeshi Takahashi, and Daisuke Inoue. 2021. Combat security alert fatigue with ai-assisted techniques. In *Proceedings of the 14th Cyber Security Experimentation and Test Workshop*. 9–16.

[5] Cisco. 2025. Snort. https://www.snort.org/.

[6] CrowdStrike. 2025. Powering the next evolution of the SOC. https://www.crowdstrike.com/en-us/platform/charlotte-ai/.

[7] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. {PentestGPT}: Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium (USENIX Security 24)*. 847–864.

[8] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 1285–1298.

[9] Chongzhou Fang, Ning Miao, Shaurya Srivastav, Jialin Liu, Ruoyu Zhang, Ruijie Fang, Ryan Tsang, Najmeh Nazari, Han Wang, Houman Homayoun, et al. 2024. Large language models for code analysis: Do {LLMs} really do their job?. In *33rd USENIX Security Symposium (USENIX Security 24)*. 829–846.

[10] Mohamed Amine Ferrag, Fatima Alwahedi, Ammar Battah, Bilel Cherif, Abdechakour Mechri, and Norbert Tihanyi. 2024. Generative ai and large language models for cyber security: All insights you need. *Available at SSRN 4853709* (2024).

[11] Google. 2023. Supercharging security with generative AI. https://cloud.google.com/blog/products/identity-security/rsa-google-cloud-security-ai-workbench-generative-ai/.

[12] Martin Husák, Martin Žádník, Václav Bartoš, and Pavol Sokol. 2020. Dataset of intrusion detection alerts from a sharing platform. *Data in Brief* 33 (2020), 106530.

[13] Hamed Jelodar, Samita Bai, Parisa Hamedi, Hesamodin Mohammadian, Roozbeh Razavi-Far, and Ali Ghorbani. 2025. Large Language Model (LLM) for Software Security: Code Analysis, Malware Analysis, Reverse Engineering. *arXiv preprint arXiv:2504.07137* (2025).

[14] Fujiao Ji, Kiho Lee, Hyungjoon Koo, Wenhao You, Euijin Choo, Hyoungshick Kim, and Doowon Kim. 2024. Evaluating the effectiveness and robustness of visual similarity-based phishing detection models. *arXiv preprint arXiv:2405.19598* (2024).

[15] Varun Badrinath Krishna. 2024. AttackQA: Development and Adoption of a Dataset for Assisting Cybersecurity Operations using Fine-tuned and Open-Source LLMs. *arXiv preprint arXiv:2411.01073* (2024).

[16] Max Landauer, Florian Skopik, and Markus Wurzenberger. 2024. Introducing a new alert data set for multi-step attack analysis. In *Proceedings of the 17th cyber security experimentation and test workshop*. 41–53.

[17] Zhanshi Li, Tong Li, Runzi Zhang, Di Wu, and Zhen Yang. 2022. A Novel Network Alert Classification Model based on Behavior Semantic.. In *SEKE*. 553–558.

[18] Xihuan Lin, Jie Zhang, Gelei Deng, Tianzhe Liu, Xiaolong Liu, Changcai Yang, Tianwei Zhang, Qing Guo, and Riqing Chen. 2025. IRCopilot: Automated Incident Response with Large Language Models. *arXiv preprint arXiv:2505.20945* (2025).

[19] Joseph Muniz, Gary McIntyre, and Nadhem AlFardan. 2015. *Security operations center: Building, operating, and maintaining your SOC*. Cisco Press.

[20] OISF. 2025. Suricata. https://suricata.io/.

[21] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL]

[22] Shuva Paul, Farhad Alemi, and Richard Macwan. 2025. LLM-Assisted Proactive Threat Intelligence for Automated Reasoning. *arXiv preprint arXiv:2504.00428* (2025).

[23] Miloslova Plachkinova and Chris Maurer. 2018. Security breach at target. *Journal of Information Systems Education* 29, 1 (2018), 11–20.

[24] OWASP Modsecurity Project. 2025. ModSecurity. https://modsecurity.org/.

[25] Microsoft Security. 2025. Microsoft Security Copilot. https://www.microsoft.com/en-us/security/business/ai-machine-learning/microsoft-security-copilot/.

[26] Xiangmin Shen, Lingzhi Wang, Zhenyuan Li, Yan Chen, Wencheng Zhao, Dawei Sun, Jiashui Wang, and Wei Ruan. 2024. Pentestagent: Incorporating llm agents to automated penetration testing. *arXiv preprint arXiv:2411.05185* (2024).

[27] Ali Shiravi, Hadi Shiravi, Mahbod Tavallaee, and Ali A Ghorbani. 2012. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *computers & security* 31, 3 (2012), 357–374.

[28] Xiaokui Shu, Ke Tian, Andrew Ciambrone, and Danfeng Yao. 2017. Breaking the target: An analysis of target data breach and lessons learned. *arXiv preprint arXiv:1701.04940* (2017).

[29] Sanchana Srikanth, Mohammad Hasanuzzaman, and Farah Tasnur Meem. 2024. Evaluating the usability of llms in threat intelligence enrichment. *arXiv preprint arXiv:2409.15072* (2024).

[30] Norbert Tihanyi, Mohamed Amine Ferrag, Ridhi Jain, Tamas Bisztray, and Merouane Debbah. 2024. CyberMetric: a benchmark dataset based on retrieval-augmented generation for evaluating LLMs in cybersecurity knowledge. In *2024 IEEE International Conference on Cyber Security and Resilience (CSR)*. IEEE, 296–302.

[31] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[32] Thijs Van Ede, Hojjat Aghakhani, Noah Spahn, Riccardo Bortolameotti, Marco Cova, Andrea Continella, Maarten Van Steen, Andreas Peter, Christopher Kruegel, and Giovanni Vigna. 2022. Deepcase: Semi-supervised contextual analysis of security events. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 522–539.

[33] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[34] Manfred Vielberth, Fabian Böhm, Ines Fichtinger, and Günther Pernul. 2020. Security operations center: A systematic study and open challenges. *Ieee Access* 8 (2020), 227756–227779.

[35] Dawei Wang, Geng Zhou, Xianglong Li, Yu Bai, Li Chen, Ting Qin, Jian Sun, and Dan Li. 2025. The Digital Cybersecurity Expert How Far Have We Come. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3273–3290.

[36] Xiaoyu Wang, Xiaobo Yang, Xueping Liang, Xiu Zhang, Wei Zhang, and Xiaorui Gong. 2024. Combating alert fatigue with alertpro: Context-aware alert prioritization using reinforcement learning for multi-step attack detection. *Computers & Security* 137 (2024), 103583.

[37] HanXiang Xu, ShenAo Wang, Ningke Li, Kailong Wang, Yanjie Zhao, Kai Chen, Ting Yu, Yang Liu, and HaoYu Wang. 2024. Large language models for cyber security: A systematic literature review. *arXiv preprint arXiv:2405.04760* (2024).

[38] Zhengmin Yu, Jiutian Zeng, Siyi Chen, Wenhan Xu, Dandan Xu, Xiangyu Liu, Zonghao Ying, Nan Wang, Yuan Zhang, and Min Yang. 2024. CS-Eval: A Comprehensive Large Language Model Benchmark for CyberSecurity. *arXiv preprint arXiv:2411.16239* (2024).

[39] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* 1, 2 (2023).

[40] M Zissman. 2000. DARPA intrusion detection scenario specific data sets. *Lincoln Laboratory, Massachusetts Institute of Technology, Lexington* (2000).

A OPEN SCIENCE

To support the evaluation of our core contributions, we will provide the following artifacts through an anonymous repository during double-blind review: (1) representative raw SOC alert examples from each of the three participating enterprises; (2) the processed SecAlertBenchdataset used in our evaluation; (3) evaluation scripts for prompt construction, model output parsing, metric computation, and configuration control; and (4) the evaluation results reported in this paper. These artifacts allow reviewers to inspect the benchmark construction process, reproduce the main evaluation pipeline, and verify the reported results. The repository will be made available at https://anonymous.4open.science/r/SecAlertBench during review.

B ETHICAL CONSIDERATIONS

This study utilizes real-world network alert logs collected from production SOC environments, which may contain sensitive organizational or user-related information. To minimize potential harm, we applied targeted anonymization before annotation and evaluation. Sensitive values that directly exposed private information, such as plaintext passwords, credential-like strings, or enterprise-specific identifiers, were replaced with placeholders. At the same time, we preserved the surrounding alert context, rule semantics, protocol

metadata, and payload structure whenever they were necessary for security analysis and label determination. We avoided excessive sanitization, as removing too much contextual evidence could weaken the distinction between malicious activity and benign business traffic. All data handling procedures followed the data governance requirements of the participating enterprises to protect sensitive information while maintaining the benchmark's utility for realistic Tier-1 alert triage evaluation. Furthermore, data access was strictly limited to authorized researchers under formal confidentiality agreements to prevent misuse.

C ADDITIONAL DATASET STATISTICS

In this section, we provide a complete example of a preprocessed alert and additional dataset statistics to complement the summary in § 3.4.

C.1 An Example of a Preprocessed Alert

Table 12 shows a representative preprocessed alert in our benchmark. The example preserves the core evidence required for Tier-1 alert triage, including the rule name, network metadata, request and response content, and payload semantics.

Table 12: An example of a preprocessed alert.

TIMESTAMP	**SIP**	**DIP**	**PROTO**
1747037759	10.16.3.237	10.115.50.29	http
RULE NAME		**SPORT**	**DPORT**
Directory traversal attack		46144	8081
PACKAGE DATA			
REQUEST HEADER		**REQUEST BODY**	
GET /manage/log/view? filename=/etc/passwd &base=../../../../../../../../ HTTP/1.1		N/A	
RESPONSE HEADER		**RESPONSE BODY**	
HTTP/1.1 400 Bad Request Content-Type: text/plain; charset=utf-8		400 Bad Request	
PAYLOAD			
filename=/etc/passwd&base=.. /../../../../../../../			
LABEL			
Attack			

C.2 Additional Dataset Statistics

To further characterize the data distribution of SecAlertBench, Figure 5 presents several supplementary statistics. These plots illustrate the class distribution, source and destination IP diversity, payload length characteristics, lexical patterns in alert payloads, the long-tail distribution of rule names, and the composition of high-level alert types.

Several observations can be made from Figure 5. First, the benchmark is moderately imbalanced, with Non-Attack alerts outnumbering Attack alerts, which reflects the practical alert distribution in real SOC environments. Second, the SIP and DIP distributions show clear diversity and long-tail characteristics, indicating that the dataset covers a wide range of communicating hosts rather than being dominated by only a few endpoints. Third, the payload length distribution also exhibits substantial variation, and the box plot suggests that Non-Attack alerts often contain longer payloads, which is consistent with the fact that benign business traffic may include richer textual content. Fourth, the rule-name distribution is strongly long-tailed, showing that the benchmark contains both frequent and low-frequency alert rules. Finally, the high-level alert-type statistics further confirm that the dataset covers diverse security scenarios, and that the ratio between Attack and Non-Attack differs across alert types, which makes Tier-1 alert triage a non-trivial classification problem.

D DETAILED PROMPTING STRATEGIES

This section provides the detailed prompt templates used in the configuration sensitivity analysis. As shown in Table 13, we evaluate five prompting strategies: Zero-Shot, User-baseline, One-shot, Few-shot, and CoT. The Zero-Shot prompt serves as the baseline, where the model is given only the task description, label definitions, and output constraint. The User-baseline prompt simulates a more guided interaction by adding a role setting, task confirmation, and strengthened decision constraints. The One-shot and Few-shot prompts provide labeled reference alerts before the target alert, allowing us to examine whether demonstrations help calibrate the model's triage decisions. Finally, the CoT prompt requires the model to produce a concise reasoning process before the final label, which enables us to further analyze the alignment between reasoning and classification decisions.

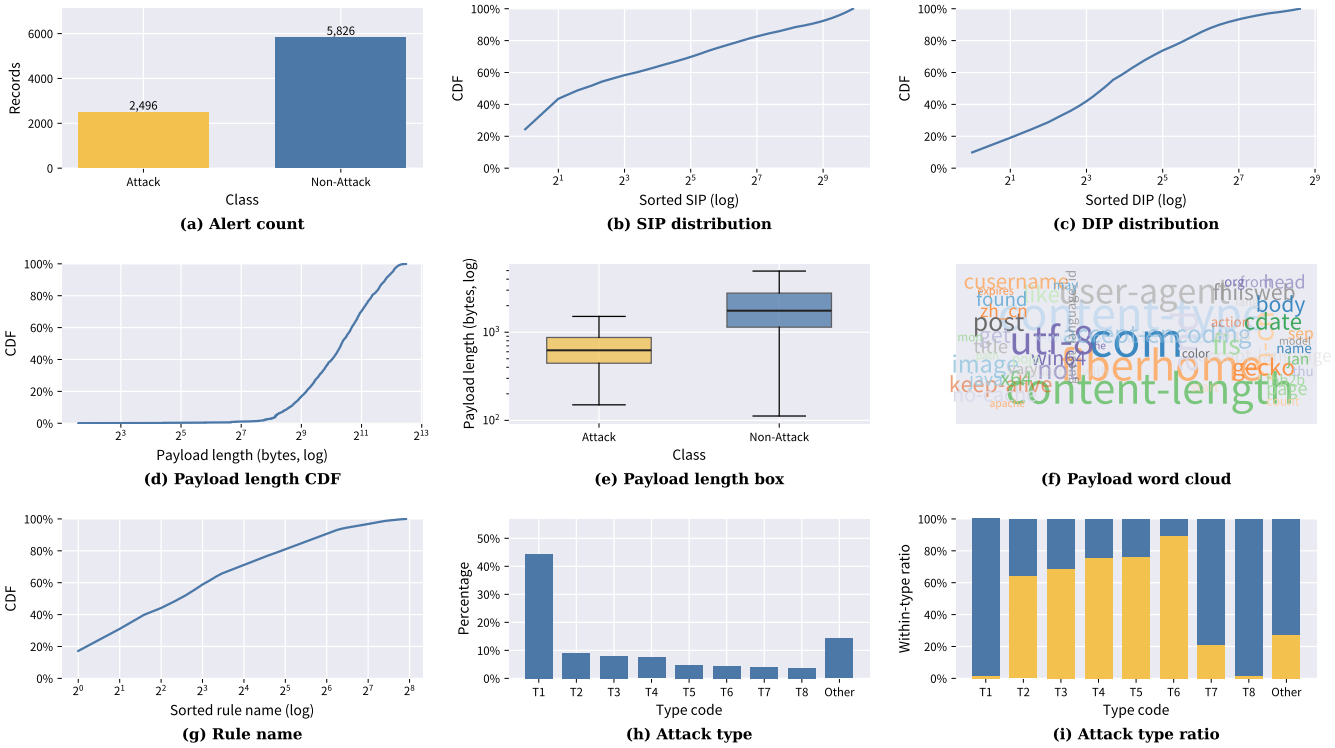

Figure 5: Additional statistics of SecAlertBench. (a) Alert count: the number of Attack and Non-Attack samples. (b) SIP distribution: the cumulative distribution of source IPs. (c) DIP distribution: the cumulative distribution of destination IPs. (d) Payload length CDF: the cumulative distribution of payload lengths. (e) Payload length box: the payload length distributions of Attack and Non-Attack alerts. (f) Payload word cloud: the most frequent words in alert payloads. (g) Rule name: the long-tail distribution of alert rule names. (h) Attack type: the percentage of different high-level alert types, where T1/T2/... denote alert-type groups sorted by frequency. (i) Attack type ratio: the within-type ratio of Attack and Non-Attack samples for each high-level alert type.

Table 13: Prompting strategies used in the configuration sensitivity analysis.

Name	Template & Example
Zero-Shot	**SYSTEM** <task description> <label definition> <output constraint> **USER** <target alert record>
	SYSTEM You will be given one network alert record and your task is to classify the alert into one of two labels: Attack or Non-Attack. Attack indicates real malicious activity, attack attempts, or other security-threatening actions. Non-Attack indicates benign traffic, normal operations, false positives, or informational events. Do not output explanations or any additional text. Return only "Attack" or "Non-Attack". **USER** Alert record (JSON): <normalized alert record>
User -baseline	**SYSTEM** <role setting> **USER** <task definition> **ASSISTANT** <task confirmation> **USER** <strengthened constraints> **ASSISTANT** <constraint confirmation> **USER** <positive start> <target alert record>
	SYSTEM You are an experienced Tier-1 SOC alert triage analyst. Your responsibility is to make accurate and conservative alert classification decisions. **USER** Task definition: You will be given one network alert record and must classify it into exactly one of two labels: Attack or Non-Attack. **ASSISTANT** Confirmed. I understand the task definition. **USER** Strengthened constraints: Base your decision only on the provided alert record. Do not output explanations, analysis, or any extra text. Output must be exactly one label. If evidence is insufficient for a real attack, choose Non-Attack. **ASSISTANT** Confirmed. I will strictly follow all constraints and only output one final label. **USER** Great, start now and classify the following alert. Alert record (JSON): <normalized alert record>
One-shot	**SYSTEM** <task description> <label definition> <output constraint> **USER** <labeled demonstration> <target alert record>
	SYSTEM You will be given one labeled reference alert and one target alert. Use the reference as guidance, then classify the target alert into one of two labels: Attack or Non-Attack. Do not output explanations or any additional text. Return only "Attack" or "Non-Attack". **USER** Reference Example: <normalized alert record> Reference Label: <Attack or Non-Attack> Now classify the following target alert. Target Alert record (JSON): <normalized alert record>
Few-shot	**SYSTEM** <task description> <label definition> <output constraint> **USER** <labeled demonstrations> <target alert record>
	SYSTEM You will be given multiple labeled reference alerts and one target alert. Use the references as guidance, then classify the target alert into one of two labels: Attack or Non-Attack. Do not output explanations or any additional text. Return only "Attack" or "Non-Attack". **USER** Reference Example 1: <normalized alert record> Reference Label 1: Attack ### Reference Example 2: <normalized alert record> Reference Label 2: Non-Attack ### Reference Example 3: <normalized alert record> Reference Label 3: <Attack or Non-Attack> ### ... Now classify the following target alert. Target Alert record (JSON): <normalized alert record>
CoT	**SYSTEM** <task description> <label definition> <reasoning requirement> <output format> **USER** <target alert record>
	SYSTEM You will be given one network alert record and your task is to classify the alert into one of two labels: Attack or Non-Attack. Before giving the final label, first output a concise reasoning process in no more than 200 words. Then output the final label. The final label must be exactly one of "Attack" or "Non-Attack". Output format: Reasoning: <reasoning process>; Label: <Attack or Non-Attack>. **USER** Alert record (JSON): <normalized alert record>